

```
In [ ]: import marimo as mo
```

```
In [ ]: # https://claude.ai/code/session_01UKPx4PhmMbK1saW2Rvzkk
import anywidget, traitlets

class Clickable(anywidget.AnyWidget):
    _esm = """
    function render({ model, el }) {
        el.innerHTML = model.get("html");
        el.style.cursor = "pointer";
        el.style.display = "inline";
        el.onclick = () => { model.set("clicks", model.get("clicks") + 1);
model.save_changes(); };
    }
    export default { render };
    """

    html = traitlets.Unicode("").tag(sync=True)
    clicks = traitlets.Int(0).tag(sync=True)

w = mo.ui.anywidget(Clickable(html="<b style=\"display: inline\">clique
aqui</b>"))
```

Horário escolar

Para gerar um horário com base nos dados em `./dados/`, .

```
In [ ]: from pathlib import Path
import csv

# data folders are relative to this notebook, not to where marimo was
# launched
root = mo.notebook_dir() or Path('.')

# https://claude.ai/code/session_01F7LZyQp7Dv7GgNpcerp6aS
def get_data(from_folder):
    data = {}
    for file in (root / from_folder).rglob('*.csv'):
        with file.open(encoding='utf-8', newline='') as f:
            data[file.stem] = list(csv.DictReader(f))
    return data
```

```
In [ ]: # https://claude.ai/code/session_01loP4T7nB5aq5sgZ9XxvDvf
def assert_data(from_folder, data):
    expected = {'disciplinas', 'disponibilidade_excecoes', 'salas',
'turmas'}
    # bonus: teacher preferences are optional
    optional = {'preferencias'}
    unknown = sorted(data.keys() - expected - optional)
    if unknown:
```

```

        raise ValueError(
            f"Ficheiros CSV não reconhecidos em {from_folder}: {'',
            '.join(unknown)}. "
            f"Esperados: {'', '.join(sorted(expected))}; opcional: {'',
            '.join(sorted(optional))}."
        )

    missing = sorted(expected - data.keys())
    if missing:
        raise ValueError(
            f"Ficheiros CSV em falta: {missing}"
        )

    # R4: duplo_perodo subjects are split into blocks of 2 periods
    odd = [subject['disciplina'] for subject in data['disciplinas']
           if subject['duplo_perodo'] == 'sim' and
           int(subject['carga_semanal']) % 2]
    if odd:
        raise ValueError(
            f"Disciplinas com duplo_perodo=sim e carga_semanal ímpar: {'',
            '.join(odd)}."
        )

    # R6 and preferences: each row names one of the 5 × 5 periods of the
    week
    for name in ('disponibilidade_excecoes', 'preferencias'):
        invalid = [f"{row['professor']} ({row['dia']}, {row['periodo']})"
                   for row in data.get(name, [])
                   if row['dia'] not in ('Seg', 'Ter', 'Qua', 'Qui', 'Sex')
                   or row['periodo'] not in ('1', '2', '3', '4', '5')]
        if invalid:
            raise ValueError(
                f"{name}.csv com dia/periodo inválido: {'',
                '.join(invalid)}."
            )

    # R7: every sala_especial must be a special room of salas.csv
    special = {room['sala'] for room in data['salas'] if room['tipo'] ==
    'especial'}
    missing_rooms = sorted({subject['sala_especial'] for subject in
    data['disciplinas']
                           if subject['sala_especial'] and
    subject['sala_especial'] not in special})
    if missing_rooms:
        raise ValueError(
            f"Salas especiais em falta em salas.csv: {'',
            '.join(missing_rooms)}."
        )

```

```

In [ ]: from dataclasses import dataclass
        from collections import namedtuple
        from html import escape

        DAYS = ['Seg', 'Ter', 'Qua', 'Qui', 'Sex']
        # every (day, slot) of the week: 5 days × 5 slots
        TIMES = [(day, slot) for day in range(5) for slot in range(5)]

```

```

At = namedtuple('At', 'day slot')

# one period of one subject for one grade, e.g.
# Lecture(at=At(day=0, slot=1), subject='Matemática', professor='Prof.
Ana', grade='7ºA')
@dataclass
class Lecture:
    at: At
    subject: str
    professor: str
    grade: str
    room: str = None

# https://claude.ai/code/session_01LGQikCyDfhpbSp5UqZvVCw
class Timetable:
    CSS = """
        .timetables {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(min(100%,
30rem), 1fr));
            gap: 1.5rem;
            margin-top: 1rem;
            hyphens: auto;
            overflow-wrap: break-word;
        }
        .timetables table {
            width: 100%;
            table-layout: fixed;
            border-collapse: separate;
            border-spacing: 3px;
            font-size: 0.9rem;
        }
        .timetables caption {
            text-align: left;
            font-size: 1rem;
            font-weight: 600;
            padding: 0 3px 0.25rem;
        }
        .timetables th {
            font-weight: 500;
            opacity: 0.6;
            text-align: left;
            padding: 0.25rem 0.5rem;
        }
        .timetables tr > th:first-child {
            width: 2.25rem;
            padding: 0;
            text-align: center;
        }
        .timetables td {
            height: 2.5rem;
            padding: 0.25rem 0.5rem;
            border-radius: 6px;
            line-height: 1.2;
            background: color-mix(in srgb, currentColor 4%, transparent);
    """

```

```

    }
    .timetables td:not(:empty) {
        font-weight: 500;
        background: oklch(0.7 0.14 var(--hue) / 0.2);
        box-shadow: inset 3px 0 oklch(0.65 0.15 var(--hue));
    }
    .timetables td.changed {
        outline: 2px dashed oklch(0.6 0.15 var(--hue));
        outline-offset: -2px;
    }
    .timetables small {
        display: block;
        font-weight: 400;
        opacity: 0.7;
    }
}
}

def __init__(self, lectures, seconds=None):
    self.lectures = lectures
    # how long the solver took to find it
    self.seconds = seconds
    # R9: when set, lectures that are not in `previous` (same time and
    room) are outlined
    self.previous = None

    # Uma tabela por turma, com os tempos nas linhas e os dias nas colunas
    def _repr_html_(self):
        subjects = dict.fromkeys(lecture.subject for lecture in
self.lectures)
        # each subject keeps the same colour across grades
        hues = {subject: 360 * i // len(subjects) for i, subject in
enumerate(subjects)}
        tables = []
        for grade in dict.fromkeys(lecture.grade for lecture in
self.lectures):
            cells = {lecture.at: lecture for lecture in self.lectures if
lecture.grade == grade}
            rows = ''
            for slot in range(5):
                rows += f'<tr><th>{slot + 1}</th>'
                for day in range(5):
                    lecture = cells.get((day, slot))
                    if lecture is None:
                        rows += '<td></td>'
                        continue
                    room = f'<small>{escape(lecture.room)}</small>' if
lecture.room else ''
                    changed = (' class="changed"' if self.previous is not
None
                                and lecture not in self.previous.lectures
                                else '')
                    rows += (f'<td style="--hue: {hues[lecture.subject]}"
{changed}>'
                            f'{escape(lecture.subject)}{room}</td>')
                rows += '</tr>'
            header = ''.join(f'<th>{day}</th>' for day in DAYS)

```

```

        tables.append(f'<table><caption>Turma {escape(grade)}</caption>'
                        f'<tr><th></th>{header}</tr>{rows}</table>')
    return f'<style>{self.css}</style><div class="timetables"
    lang="pt">{''.join(tables)}</div>'

```

```

In [ ]: from ortools.sat.python import cp_model

# R6: the (professor, day, slot) in which a teacher can't teach
def unavailable_periods(data):
    return {(row['professor'], DAYS.index(row['dia']), int(row['periodo'])
    - 1)
            for row in data['disponibilidade_excecoes']}

# R7: a subject uses its sala_especial or, when that is empty, the normal
room
def uses(subject, room):
    if subject['sala_especial']:
        return subject['sala_especial'] == room['sala']
    return room['tipo'] == 'normal'

def build_model(data):
    model = cp_model.CpModel()
    grades = [grade['turma'] for grade in data['turmas']]
    subjects = data['disciplinas']
    teachers = dict.fromkeys(subject['professor'] for subject in subjects)
    unavailable = unavailable_periods(data)

    # x[grade, subject, day, slot] is true when the grade has that subject
    at that time
    x = {(grade, subject['disciplina'], day, slot):
         model.new_bool_var(f"
{grade}_{subject['disciplina']}_{day}_{slot}")
        for grade in grades for subject in subjects for day, slot in
TIMES}

    for grade in grades:
        # R1: no two lectures of the same grade at the same time
        for day, slot in TIMES:
            model.add(sum(x[grade, subject['disciplina'], day, slot] for
subject in subjects) <= 1)

        for subject in subjects:
            name = subject['disciplina']
            # R2: exactly carga_semanal lectures a week
            model.add(sum(x[grade, name, day, slot] for day, slot in TIMES)
                        == int(subject['carga_semanal']))

        for day in range(5):
            today = [x[grade, name, day, slot] for slot in range(5)]
            if subject['duplo_periodo'] == 'sim':
                # R3 and R4: at most one double block a day, so 0 or 2
                lectures (never 1)...
                model.add(sum(today) <= 2)
                model.add(sum(today) != 1)
                # R4: ... in consecutive slots: never two slots more

```

than 1 apart

```
        for slot in range(5):
            for later in range(slot + 2, 5):
                model.add(today[slot] + today[later] <= 1)
    else:
        # R3: at most one lecture of this subject a day
        model.add(sum(today) <= 1)

    # R6: no lectures when the teacher is unavailable
    for slot in range(5):
        if (subject['professor'], day, slot) in unavailable:
            model.add(today[slot] == 0)
    # R8: all of the above comes from `data`, read from the CSV files

    # the lectures `teacher` gives at (day, slot), over all grades: 0 or 1
    because of R5
    def teaching(teacher, day, slot):
        return sum(x[grade, subject['disciplina'], day, slot]
                    for grade in grades for subject in subjects
                    if subject['professor'] == teacher)

    # R5: a teacher gives at most one lecture at a time, over all grades
    for teacher in teachers:
        for day, slot in TIMES:
            model.add(teaching(teacher, day, slot) <= 1)

    # R7: at any time, never more lectures in a type of room than rooms of
    that type
    for room in data['salas']:
        for day, slot in TIMES:
            model.add(sum(x[grade, subject['disciplina'], day, slot]
                           for grade in grades for subject in subjects if
            uses(subject, room))
                       <= int(room['quantidade']))

    # 01: a slot is a hole when the teacher teaches before and after it
    that day, but not in it
    holes = []
    for teacher in teachers:
        for day in range(5):
            busy = [teaching(teacher, day, slot) for slot in range(5)]
            for slot in range(1, 4):
                hole = model.new_bool_var(f'hole_{teacher}_{day}_{slot}')
                for before in range(slot):
                    for after in range(slot + 1, 5):
                        model.add(hole >= busy[before] + busy[after] -
            busy[slot] - 1)
                holes.append(hole)

    return model, x, holes

# solves the model and reads the timetable from the x[grade, subject, day,
slot] that are true
def solve(model, x, data):
    solver = cp_model.CpSolver()
    # 01 needn't be optimal: after 10 s, keep the best timetable found so
```

```

far
    solver.parameters.max_time_in_seconds = 10
    if solver.solve(model) not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
        return None
    teachers = {subject['disciplina']: subject['professor'] for subject in
data['disciplinas']}
    timetable = Timetable([
        Lecture(at=At(day, slot), subject=subject,
professor=teachers[subject], grade=grade)
        for (grade, subject, day, slot), chosen in x.items() if
solver.value(chosen)
    ], seconds=solver.wall_time)
    assign_rooms(data, timetable)
    return timetable

# R7: each lecture goes to its subject's sala_especial or, when that is
empty, the normal room
def assign_rooms(data, timetable):
    subjects = {subject['disciplina']: subject for subject in
data['disciplinas']}
    for room in data['salas']:
        for lecture in timetable.lectures:
            if uses(subjects[lecture.subject], room):
                lecture.room = room['sala']

# H0: a timetable from scratch, with as few holes as possible (01)
def generate_timetable(data):
    model, x, holes = build_model(data)
    model.minimize(sum(holes))
    return solve(model, x, data)

# R9: H1 for `data`, starting from `previous` (H0) and moving as few of its
lectures as possible
def regenerate_timetable(previous, data):
    model, x, holes = build_model(data)
    before = {(lecture.grade, lecture.subject, lecture.at.day,
lecture.at.slot)
               for lecture in previous.lectures}
    # the search starts from H0...
    for key, chosen in x.items():
        model.add_hint(chosen, key in before)
    # ... and keeps as many of its lectures at the same time as it can
    model.maximize(sum(chosen for key, chosen in x.items() if key in
before))
    return solve(model, x, data)

```

```
In [ ]: from collections import Counter, defaultdict
```

```

# R1–R7 checked directly on the lectures of a timetable, without the model
def check_requirements(data, timetable):
    lectures = timetable.lectures
    subjects = {subject['disciplina']: subject for subject in
data['disciplinas']}
    rooms = {room['sala']: room for room in data['salas']}
    unavailable = unavailable_periods(data)

```

```

    grade_busy = Counter((lecture.grade, lecture.at) for lecture in
lectures)
    teacher_busy = Counter((lecture.professor, lecture.at) for lecture in
lectures)
    room_busy = Counter((lecture.room, lecture.at) for lecture in lectures)
    per_week = Counter((lecture.grade, lecture.subject) for lecture in
lectures)
    # the slots of each subject on each day, for each grade
    per_day = defaultdict(list)
    for lecture in lectures:
        per_day[lecture.grade, lecture.subject,
lecture.at.day].append(lecture.at.slot)

    def double(name):
        return subjects[name]['duplo_periodo'] == 'sim'

    return {
        'R1': all(n == 1 for n in grade_busy.values()),
        'R2': all(per_week[grade['turma'], subject['disciplina']] ==
int(subject['carga_semanal']))
        for grade in data['turmas'] for subject in
data['disciplinas']),
        'R3': all(len(slots) <= (2 if double(name) else 1)
        for (grade, name, day), slots in per_day.items()),
        'R4': all(len(slots) == 2 and abs(slots[0] - slots[1]) == 1
        for (grade, name, day), slots in per_day.items() if
double(name)),
        'R5': all(n == 1 for n in teacher_busy.values()),
        'R6': all((lecture.professor, lecture.at.day, lecture.at.slot) not
in unavailable
        for lecture in lectures),
        'R7': all(lecture.room in rooms and uses(subjects[lecture.subject],
rooms[lecture.room])
        for lecture in lectures)
        and all(n <= int(rooms[room]['quantidade']) for (room, at), n
in room_busy.items()),
    }

# 01 recounted on the timetable: free slots between a teacher's first and
last lecture of a day
def count_holes(timetable):
    busy = defaultdict(set)
    for lecture in timetable.lectures:
        busy[lecture.professor, lecture.at.day].add(lecture.at.slot)
    return sum(max(slots) - min(slots) + 1 - len(slots) for slots in
busy.values())

# R9: lectures of `before` that are not in `after` at the same time and in
the same room
def count_changes(before, after):
    return sum(lecture not in after.lectures for lecture in
before.lectures)

```


▼ build_model(data)

```
1 def build_model(data):
2     model = cp_model.CpModel()
3     grades = [grade['turma'] for grade in data['turmas']]
4     subjects = data['disciplinas']
5     teachers = dict.fromkeys(subject['professor'] for subject in subjects)
6     unavailable = unavailable_periods(data)
7
8     # x[grade, subject, day, slot] is true when the grade has that subject at that time
9     x = {(grade, subject['disciplina'], day, slot):
10         model.new_bool_var(f"{grade}_{subject['disciplina']}_{day}_{slot}")
11         for grade in grades for subject in subjects for day, slot in TIMES}
12
13     for grade in grades:
14         # R1: no two lectures of the same grade at the same time
15         for day, slot in TIMES:
16             model.add(sum(x[grade, subject['disciplina'], day, slot] for subject in subjects) <= 1)
17
18         for subject in subjects:
19             name = subject['disciplina']
20             # R2: exactly carga_semanal lectures a week
21             model.add(sum(x[grade, name, day, slot] for day, slot in TIMES)
22                         == int(subject['carga_semanal']))
23
24         for day in range(5):
25             today = [x[grade, name, day, slot] for slot in range(5)]
26             if subject['duplo_periodo'] == 'sim':
27                 # R3 and R4: at most one double block a day, so 0 or 2 lectures (never 1)...
28                 model.add(sum(today) <= 2)
29                 model.add(sum(today) != 1)
30                 # R4: ... in consecutive slots: never two slots more than 1 apart
31                 for slot in range(5):
32                     for later in range(slot + 2, 5):
33                         model.add(today[slot] + today[later] <= 1)
34             else:
35                 # R3: at most one lecture of this subject a day
36                 model.add(sum(today) <= 1)
37
38                 # R6: no lectures when the teacher is unavailable
39                 for slot in range(5):
40                     if (subject['professor'], day, slot) in unavailable:
41                         model.add(today[slot] == 0)
42
43     # R8: all of the above comes from `data`, read from the CSV files
44
45     # the lectures `teacher` gives at (day, slot), over all grades: 0 or 1 because of R5
46     def teaching(teacher, day, slot):
47         return sum(x[grade, subject['disciplina'], day, slot]
48                   for grade in grades for subject in subjects
49                   if subject['professor'] == teacher)
50
51     # R5: a teacher gives at most one lecture at a time, over all grades
52     for teacher in teachers:
53         for day, slot in TIMES:
54             model.add(teaching(teacher, day, slot) <= 1)
55
56     # R7: at any time, never more lectures in a type of room than rooms of that type
57     for room in data['salas']:
58         for day, slot in TIMES:
59             model.add(sum(x[grade, subject['disciplina'], day, slot]
60                           for grade in grades for subject in subjects if uses(subject, room))
61                         <= int(room['quantidade']))
62
63     # O1: a slot is a hole when the teacher teaches before and after it that day, but not in it
64     holes = []
65     for teacher in teachers:
66         for day in range(5):
67             busy = [teaching(teacher, day, slot) for slot in range(5)]
68             for slot in range(1, 4):
69                 hole = model.new_bool_var(f'hole_{teacher}_{day}_{slot}')
70                 for before in range(slot):
71                     for after in range(slot + 1, 5):
72                         model.add(hole >= busy[before] + busy[after] - busy[slot] - 1)
73                 holes.append(hole)
```

```
74     return model, x, holes
75
```

Os dados de  `dados/` são lidos e validados por `assert_data`. O modelo tem uma variável booleana `x[turma, disciplina, dia, tempo]`, verdadeira quando a turma tem essa disciplina nesse tempo.

Para cada turma, há no máximo uma aula em cada tempo (R1) e, para cada disciplina, a carga semanal é exata (R2), há no máximo uma aula por dia (R3) ou, em duplo período, 0 ou 2 tempos seguidos (R4), e nenhuma aula quando o professor está indisponível (R6).

Depois, olhando para todas as turmas juntas, um professor dá no máximo uma aula de cada vez (R5) e, em cada tempo, nenhum tipo de sala tem mais aulas do que salas (R7). O solver minimiza os buracos nos horários dos professores (O1).

Por fim, cada aula recebe a sua sala: a `sala_especial` da disciplina ou, se estiver vazia, a sala normal.

Turma 7ªA

	Seg	Ter	Qua	Qui	Sex
1º	História Sala Normal		Inglês Sala Normal	Matemática Sala Normal	
2º	Português Sala Normal	Português Sala Normal	História Sala Normal	Inglês Sala Normal	Ciências Laboratório
3º	Matemática Sala Normal		Matemática Sala Normal		
4º		Educação Física Ginásio	Ciências Laboratório	Ciências Laboratório	Português Sala Normal
5º		Educação Física Ginásio		Português Sala Normal	Matemática Sala Normal

Turma 7ªB

	Seg	Ter	Qua	Qui	Sex
1º	Ciências Laboratório	Português Sala Normal	Português Sala Normal		
2º	Matemática Sala Normal	Ciências Laboratório	Matemática Sala Normal	Matemática Sala Normal	
3º	Português Sala Normal	Matemática Sala Normal	História Sala Normal		Ciências Laboratório
4º	Educação Física Ginásio			Português Sala Normal	História Sala Normal
5º	Educação Física Ginásio	Inglês Sala Normal			Inglês Sala Normal

Construção incremental (R9)

H1 é gerado para  dados_v2/ com o mesmo modelo, mas a partir de H0:

1. H0 é dado ao solver como *hint*, isto é, como ponto de partida da procura.
2. Em vez de minimizar os buracos (O1), o solver maximiza o número de aulas de H0 que ficam no mesmo tempo (a sala depende só da disciplina, por isso também se mantém).

Como só os dados mudam, o mesmo processo serve para qualquer alteração: um professor indisponível (ou disponível) em mais tempos, uma sala avariada, uma turma nova, um professor substituído.

	Tempo	Aulas de H0 alteradas (tempo ou sala)
Incremental, a partir de H0	0.014 s	1 de 34
Do zero, sem H0	0.024 s	31 de 34

H1 incremental, com as aulas que mudaram em relação a H0 a tracejado:

Turma 7ºA

	Seg	Ter	Qua	Qui	Sex
1º	História Sala Normal		Inglês Sala Normal	Matemática Sala Normal	
2º	Português Sala Normal	Português Sala Normal	História Sala Normal	Inglês Sala Normal	Ciências Laboratório
3º	Matemática Sala Normal		Matemática Sala Normal		Matemática Sala Normal
4º		Educação Física Ginásio	Ciências Laboratório	Ciências Laboratório	Português Sala Normal
5º		Educação Física Ginásio		Português Sala Normal	

Turma 7ºB

	Seg	Ter	Qua	Qui	Sex
1º	Ciências Laboratório	Português Sala Normal	Português Sala Normal		
2º	Matemática Sala Normal	Ciências Laboratório	Matemática Sala Normal	Matemática Sala Normal	
3º	Português	Matemática	História		Ciências

	Sala Normal	Sala Normal	Sala Normal		Laboratório
4º	Educação Física Ginásio			Português Sala Normal	História Sala Normal
5º	Educação Física Ginásio	Inglês Sala Normal			Inglês Sala Normal

Verificação dos requisitos

check_requirements verifica cada horário diretamente, sem usar o modelo: relê os CSV e confirma R1–R7 aula a aula (O1 é recontado da mesma forma). dados_teste/ é um conjunto de dados diferente (3 turmas, mais uma disciplina de duplo período e a Prof. Diana indisponível às quartas): o mesmo código gera um horário válido para ele, porque nada dos dados está no código (R8).

Requisito	H0 · dados/	H1 incremental · dados_v2/	H1 do zero · dados_v2/	Teste · dados_teste/
R1 uma aula de cada vez por turma	✓	✓	✓	✓
R2 carga semanal exata	✓	✓	✓	✓
R3 no máximo uma aula por dia de cada disciplina	✓	✓	✓	✓
R4 duplo período em blocos de 2 tempos	✓	✓	✓	✓
R5 um professor numa aula de cada vez	✓	✓	✓	✓
R6 disponibilidade dos professores	✓	✓	✓	✓
R7 salas do tipo certo, sem exceder a quantidade	✓	✓	✓	✓
R8 dados lidos dos CSV de	dados/	dados_v2/	dados_v2/	dados_teste/
O1 buracos (recontados)	0	0	0	0

Horário qerado para dados teste/

